Big Data Analytics in Apache Spark

Big Data Analytics with Spark

- Spark Dataframes to work with tabular data
- Data cleaning, summary, statistics
- Spark Dataframes with SQL

Introduction to Spark Dataframes

https://spark.apache.org/docs/3.1.1/api/python/reference/api/pyspark.sql.DataFrame.html

Types of RDD: text

from local filesystem:

```
text_RDD =
sc.textFile("file:///home/cloudera/testfile1")
```

```
text_RDD.collect()
```

```
Out[]: [u'A long time ago in a galaxy far far away']
```

Types of RDD: key-value pairs

```
def split_words(line):
    return line.split()

def create_pair(word):
    return (word, 1)

pairs_RDD=text_RDD.flatMap(split_words
).map(create_pair)
```

```
pairs_RDD.collect()
Out[]: [(u'A', 1),
 (u'long', 1),
 (u'time', 1),
 (u'ago', 1),
 (u'in', 1),
 (u'a', 1),
 (u'galaxy', 1),
 (u'far', 1),
 (u'far', 1),
 (u'away', 1)]
```

Tabular dataset

Most real-world datasets have
records (rows)

each with
multiple values (columns)

Tweets

user	text	datetime	favorites	retweets
andreazonca	"spark is cool"	"2015-10-1 9:04"	5	3

Reviews

business	text	datetime	starts	user
Pan Bon	"great pizza!"	"2015-10-1 9:04"	5	andreazonca

Logs

http_code	ip	datetime	user_agent
200	127.0.0.1	"2015-10-1 9:04"	Firefox

Tabular datasets

```
students = sc.parallelize([
[100, "Alice", 8.5, "Computer Science"],
[101, "Bob", 7.1, "Engineering"],
[102, "Carl", 6.2, "Engineering"]
])
```

Mean of a column

```
def extract_grade(row):
    return row[2]

students.map(extract_grade).mean()
Out[]: 7.26666
```

Group by column

```
def extract_degree_grade(row):
    return (row[3], row[2])

degree_grade_RDD =
students.map(extract_degree_grade)
degree_grade_RDD.collect()
```

Group by column

Intermediate RDD:

```
degree_grade_RDD.collect()
```

```
Out[]:
[('Computer Science', 8.5),
 ('Engineering', 7.099999999999996),
 ('Engineering', 6.200000000000002)]
```

Group by column

Reduce by key to get the final result:

```
degree_grade_RDD.reduceByKey(max).collect()
```

```
Out[]:
```

```
[('Engineering', 7.0999999999999996),
 ('Computer Science', 8.5)]
```

Introducing Spark Dataframes

User friendly interface

Under-the-hood optimization for table-like datasets

```
students_df = sqlCtx.createDataFrame(students,
    ["id", "name", "grade", "degree"])
students_df.printSchema()
```

```
root
 |-- id: long (nullable = true)
 |-- name: string (nullable = true)
 |-- grade: double (nullable = true)
 |-- degree: string (nullable = true)
```

sqlCtx.createDataFrame?

Create a DataFrame from an RDD of tuple/list, list or pandas.DataFrame.

`schema` could be :class:`StructType` or a list of column names.

When `schema` is a list of column names, the type of each column will be inferred from `rdd`.

When `schema` is None, it will try to infer the column name and type from `rdd`, which should be an RDD of :class:`Row`, or namedtuple, or dict.

If referring needed, `samplingRatio` is used to determined how many rows will be used to do referring. The first row will be used if `samplingRatio` is None.

:param data: an RDD of Row/tuple/list/dict, list, or pandas.DataFrame
:param schema: a StructType or list of names of columns
:param samplingRatio: the sample ratio of rows used for inferring
:return: a DataFrame

```
>>> l = [('Alice', 1)]
>>> sqlCtx.createDataFrame(l).collect()
[Row(_1=u'Alice', _2=1)]
>>> sqlCtx.createDataFrame(l, ['name', 'age']).collect()
[Row(name=u'Alice', age=1)]
```

Mean of a column

```
students_df.agg({"grade": "mean"}).collect()
```

```
Out[]: [Row(AVG(grade#30)=7.2666666666666666)]
```

https://spark.apache.org/docs/3.1.1/api/python/reference/api/pyspark.sql.DataFrame.html

Group by column

```
students_df.groupBy("degree").max("grade").collect()
Out[]:
[Row(degree=u'Computer Science',
MAX(grade#30)=8.5),
 Row(degree=u'Engineering',
MAX(grade#30)=7.09999999999)]
```

Pretty print with show

```
students_df.groupBy("degree").max("grade").show()
```

```
degree                MAX(grade#30)
Computer Science      8.5
Engineering           7.1
```

Final remarks on Dataframes

- special kind of RDD
- transformations/actions/DAG work the same way
- automatic optimization to Java bytecode
- Python as fast as Scala/Java

Create Spark Dataframes

Specify a Schema

```
students_df = sqlCtx.createDataFrame(students,
    ["id", "name", "grade", "degree"]
```

```
from pyspark.sql.types import *
schema = StructType([
StructField("id", LongType(), True),
StructField("name", StringType(), True),
StructField("grade", DoubleType(), True),
StructField("degree", StringType(), True) ])

students_df = sqlCtx.createDataFrame(students, schema)
```

```
students_df.printSchema()
```

```
root
 |-- id: long (nullable = true)
 |-- name: string (nullable = true)
 |-- grade: double (nullable = true)
 |-- degree: string (nullable = true)
```

Load a JSON file

```
students_json = [
'{"id":100, "name":"Alice", "grade":8.5,
"degree":"Computer Science"}',
'{"id":101, "name":"Bob", "grade":7.1,
"degree":"Engineering"}']
with open("students.json", "w") as f:
    f.write("\n".join(students_json))
```

Dump JSON file conent

```
!cat students.json
```

```
{"id":100, "name":"Alice", "grade":8.5, "degree":"Computer Science"}
{"id":101, "name":"Bob", "grade":7.1, "degree":"Engineering"}
```

Create Dataframe with jsonFile

```
sqlCtx.jsonFile("file:///home/cloudera/students.json").show()
```

```
degree                grade  id   name
Computer Science      8.5    100  Alice
Engineering           7.1    101  Bob
```

Load sample yelp csv

```
yelp_df = sqlCtx.load(
source="com.databricks.spark.csv",
header = 'true',
inferSchema = 'true',
path =
'file:///usr/lib/hue/apps/search/examples/collections/solr_co
nfigs_yelp_demo/index_data.csv')
```

```
yelp_df.printSchema()
root
 |-- business_id: string (nullable = true)
 |-- cool: integer (nullable = true)
 |-- date: string (nullable = true)
 |-- funny: integer (nullable = true)
 |-- id: string (nullable = true)
 |-- stars: integer (nullable = true)
 |-- text: string (nullable = true)
 |-- type: string (nullable = true)
 |-- useful: integer (nullable = true)
 |-- user_id: string (nullable = true)
 |-- name: string (nullable = true)
 |-- full_address: string (nullable = true)
 |-- latitude: double (nullable = true)
 |-- longitude: double (nullable = true)
 |-- neighborhoods: string (nullable = true)
 |-- open: string (nullable = true)
 |-- review_count: integer (nullable = true)
 |-- state: string (nullable = true)
```

```
yelp_df.count()
Out[]: 1000L
```

Analytics with Dataframes on a Yelp reviews dataset

Explore the Yelp dataset

```
yelp_df = sqlCtx.load(
source='com.databricks.spark.csv',
header = 'true',
inferSchema = 'true',
path =
'file:///usr/lib/hue/apps/search/examples/collections/solr_configs_yelp_demo/index_data.csv')
```

Reference a column

As attribute:

```
yelp_df.useful
        Out[]: Column<useful>
```

As key:

```
yelp_df["useful"]
        Out[]: Column<useful>
```

Filtering

```
yelp_df.filter(yelp_df.useful >= 1).count()

yelp_df.filter(yelp_df["useful"] >= 1).count()

yelp_df.filter("useful >= 1").count()
```

```
Out[]: 601L
```

select

```
yelp_df["useful"].agg({"useful":"max"}).collect()
```

```
Out[]: AttributeError: 'Column' object has no attribute 'agg'
```

```
yelp_df.select("useful")
```

```
Out[]: DataFrame[useful: int]
```

```
yelp_df.select("useful").agg({"useful":"max"}).collect()
```

```
Out[]: [Row(MAX(useful#267)=28)]
```

Create a modified DataFrame

Rescale the useful column from 0-28 to 0-100.

Create a 2 columns DataFrame

```
yelp_df.select("id", "useful").take(5)
```

```
[Row(id=u'fWKvX83p0-ka4JS3dc6E5A', useful=5),
 Row(id=u'IjZ33sJrzXqU-0X6U8NwyA', useful=0),
 Row(id=u'IESLBzqUCLdSzSqm0eCSxQ', useful=1),
 Row(id=u'G-WvGaISbqqaMHlNnByodA', useful=2),
 Row(id=u'1uJFq2r5QfJG_6ExMRCaGw', useful=0)]
```

Modify column

```
yelp_df.select("id", yelp_df.useful/28*100).show(5)
```

```
+--------------------+-------------------+
|                  id|((useful / 28) * 100)|
+--------------------+-------------------+
|fWKvX83p0-ka4JS3d...| 17.857142857142858|
|ljZ33sJrzXqU-0X6U...|                0.0|
|lESLBzqUCLdSzSqm0...| 3.571428571428571|
|G-WvGaISbqqaMHlNn...| 7.142857142857142|
|1uJFq2r5QfJG_6ExM...|                0.0|
+--------------------+-------------------+
```

Cast (truncate) to integer

```
yelp_df.select("id",
(yelp_df.useful/28*100).cast("int")).show(5)
```

```
+--------------------+-----------------------------------------+
|                  id| CAST(((useful / 28) * 100)), IntegerType)|
+--------------------+-----------------------------------------+
|fWKvX83p0-ka4JS3d...|                                       17|
|IjZ33sJrzXqU-0X6U...|                                        0|
|lESLBzqUCLdSzSqm0...|                                        3|
|G-WvGaISbqqaMHlNn...|                                        7|
|1uJFq2r5QfJG_6ExM...|                                        0|
+--------------------+-----------------------------------------+
```

Save as new dataframe

```
useful_perc_data = yelp_df.select(
    "id",
    (yelp_df.useful/28*100).cast("int")
)
```

```
useful_perc_data.columns
```

```
Out[]: [u'id', u'CAST(((useful / 28) * 100), IntegerType)']
```

alias - rename a column

```
useful_perc_data = yelp_df.select(
    "id",
    (yelp_df.useful/28*100).cast("int").alias("useful_perc")
)
```

```
useful_perc_data.columns
```

```
Out[]: [u'id', u'useful_perc']
```

alias - rename a column

```
useful_perc_data = yelp_df.select(
    "id",
    (yelp_df.useful/28*100).cast("int").alias("useful_perc")
)
```

```
useful_perc_data.columns
```

```
Out[]: [u'id', u'useful_perc']
```

alias - rename also id

```
useful_perc_data = yelp_df.select(
    yelp_df["id"].alias("uid"),
    (yelp_df.useful/28*100).cast("int").alias("useful_perc")
)
```

```
useful_perc_data.columns
```

```
Out[]: [u'uid', u'useful_perc']
```

Ordering by column

Import functions for ascending/descending order:

```
from pyspark.sql.functions import asc, desc
```

order by usefulness

```
useful_perc_data = yelp_df.select(
    yelp_df["id"].alias("uid"),
    (yelp_df.useful/28*100).cast("int").alias("useful_perc")
).orderBy(desc("useful_perc"))
```

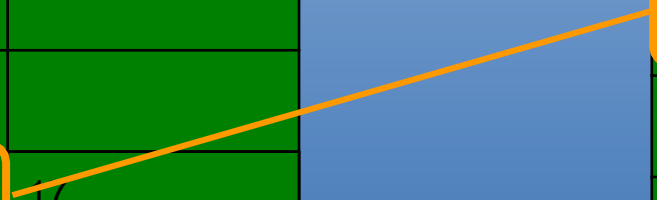
```
useful_perc_data.show(2)
```

```
uid                     useful_perc
RqwFPp_qPu-1h87pG... 100
YAXPKM-Hck6-mjF74... 82
```

Join inputs

id	useful_perc
9yKzy9PApe	17

id	review_count	state
9yKzy9PApe	6	"CA"



Join results

id	useful_perc	review_count
9yKzy9PApe	17	6

Join

```
useful_perc_data.join(
    yelp_df,
    yelp_df.id == useful_perc_data.uid,
    "inner"
)
```

Join - select

```
useful_perc_data.join(
    yelp_df,
    yelp_df.id == useful_perc_data.uid,
    "inner"
).select(useful_perc_data.uid, "useful_perc", "review_count")
```

Join - select - show

```
useful_perc_data.join(
    yelp_df,
    yelp_df.id == useful_perc_data.uid,
    "inner"
).select(useful_perc_data.uid, "useful_perc",
"review_count").show(5)
```

Output dataset

uid	useful_perc	review_count
WRBYytJAaJI1BTQG5...	71	362
GXj4PNAi095-q9ynP...	3	76
1sn0-eY_d1Dhr6Q2u...	0	9
MtFe-FuiOmo0vlo16...	0	7
EMYmuTIyeNBy5QB9P...	7	19

Cache in memory

```
useful_perc_data.join(
    yelp_df,
    yelp_df.id == useful_perc_data.uid,
    "inner"
).cache().select(useful_perc_data.uid, "useful_perc",
"review_count").show(5)
```

Run it again!

Completed DataFrames

- Completed analytics with DataFrames
- Next we'll focus on interoperability with SQL query language and Hive